

UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: Básica

Campo de Estudio: Tronco común

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO- EXPERIMENTAL

Nombre(s) del(de los) Estudiante(s): MIGUEL CAHUASQUI	
Asignatura: Estructura de Datos	Calificación:
Fecha del informe: 19 de julio de 2026	
Nº Práctica: 02	
Título de la Práctica: Implementación de Pilas - Botón retroceder	

Introducción	El diseño de elementos de software con tecnologías basadas en componentes requiere estructuras eficientes para el manejo temporal de la información. En el ecosistema de C#, las estructuras de datos, como las pilas (Stacks), proporcionan mecanismos robustos para construir aplicaciones eficientes. Esta práctica modela el comportamiento del botón “retroceder” de un navegador web mediante el principio LIFO (Last-In, First-Out), el cual garantiza que el último elemento ingresado sea el primero en ser recuperado. La implementación se desarrolla utilizando Programación Orientada a Objetos (POO), permitiendo analizar tanto el comportamiento de la estructura como la eficiencia y los algoritmos subyacentes.
---------------------	--

Desarrollo o procedimiento	Para resolver la problemática del historial del navegador, se diseñó un sistema en lenguaje C# bajo el paradigma de Programación Orientada a Objetos (POO). El entorno de desarrollo utilizado fue de escritorio. 1. Clase PaginaWeb: Contiene los atributos tipados <i>Url</i> y <i>Titulo</i> para representar la información de cada sitio visitado. 2. Clase Navegador: Contiene la lógica principal. Utiliza la estructura nativa <i>Stack<PaginaWeb></i> del espacio de nombres <i>System.Collections.Generic</i> para almacenar el historial de páginas anteriores. 3. Métodos implementados: <i>VisitarPagina()</i> apila la página actual; <i>Retroceder()</i> extrae el último elemento mediante <i>Pop()</i>; <i>MostrarHistorial()</i> itera sobre la pila para visualizar los elementos sin alterar su estado original (Reportería). 4. Análisis de tiempo: Se importó la librería <i>System.Diagnostics</i> y se implementó la clase <i>Stopwatch</i> para medir el tiempo exacto de ejecución. Agente de IA utilizado: Para la estructuración del proyecto y validación de las clases en C#, se utilizó el modelo Gemini de Google (estimando su asistencia en un 30 % del código y 100 % en la recopilación bibliográfica), mientras que el diseño y pruebas locales se realizaron manualmente.
Resultados	La implementación del sistema logró modelar exitosamente la estructura LIFO. Al simular la navegación secuencial, el sistema apiló los objetos correctamente. • En la reportería, se verificó que la última página visitada ocupaba el tope de la pila. • Al ejecutar el método de retroceso, el sistema desencadenó correctamente la salida a la página anterior sin pérdidas de memoria ni errores de referencias nulas. • El análisis algorítmico demostró tiempos de ejecución mínimos (inferiores a 10 ms para las operaciones locales medidas con Stopwatch), confirmando la eficiencia de las estructuras dinámicas frente a arreglos estáticos.
Conclusiones	La estructura de datos Pila (Stack) es idónea para simular un historial de navegación web debido a su comportamiento natural LIFO, cumpliendo con el objetivo central de la práctica. • Ventajas: Las pilas ofrecen una complejidad computacional de $O(1)$ para las operaciones de inserción y extracción. El navegador puede retroceder de página de manera instantánea sin importar si el historial tiene 5 o 500 páginas almacenadas.

	• Desventajas: La principal limitación de la pila pura es que no permite el acceso aleatorio. Para acceder a una página visitada hace varios saltos sin desapilar (y perder) los datos intermedios, se requerirían estructuras de soporte adicionales.
Referencias Bibliográficas	Castillo Barrios, J. I. (2022). <i>Método adaptivo-reactivo de replicación para sistemas de almacenamiento de alta disponibilidad</i> [Tesis de maestría, Centro de Investigación y de Estudios Avanzados del Instituto Politécnico Nacional]. Repositorio Institucional. García, B. C. (2025). <i>Diseño de elementos software con tecnologías basadas en componentes</i>. UF1289. TUTOR FORMACIÓN. Jamro, M. (2024). <i>C# Data Structures and Algorithms: Harness the power of C# to build a diverse range of efficient applications</i>. Packt Publishing Ltd. Jiménez Suárez, M. A. (2025). <i>Análisis de algoritmos para la generación de trayectorias SAR con IA</i> [Trabajo de fin de grado]. Repositorio Institucional.
Anexos	Repositorio de Código: https://github.com/Rodrigo2461/PE-ESTRUCTURA-DE-DATOS. git

Anexo: Código Fuente en C# (Program.cs)

```
1 using System;
2 using System.Collections.Generic;
3 using System.Diagnostics;
4
5 namespace SimulacionNavegador
6 {
7     // Clase que representa el objeto individual (Sitio web)
8     public class PaginaWeb
9     {
10         public string Url { get; set; }
11         public string Titulo { get; set; }
12
13         public PaginaWeb(string url, string titulo)
14         {
15             Url = url;
16             Titulo = titulo;
17         }
18
19         public override string ToString()
20         {
21             return $"{Titulo} ({Url})";
22         }
23     }
24
25     // Clase principal con la lógica de estructura de datos (Pila)
26     public class Navegador
27     {
28         private Stack<PaginaWeb> historialAtras;
29         public PaginaWeb PaginaActual { get; private set; }
30
31         public Navegador()
32         {
33             historialAtras = new Stack<PaginaWeb>();
34             PaginaActual = null;
35         }
36
37         public void VisitarPagina(PaginaWeb nuevaPagina)
38         {
39             if (PaginaActual != null)
40             {
41                 historialAtras.Push(PaginaActual);
42             }
43             PaginaActual = nuevaPagina;
44             Console.WriteLine($"[Navegando] -> {PaginaActual}");
45         }
46
47         public void Retroceder()
48         {
49             if (historialAtras.Count > 0)
50             {
51                 PaginaWeb paginaAnterior = historialAtras.Pop();
52                 Console.WriteLine($"\\n[Retrocediendo] <- Saliendo de {PaginaActual.
53                     Titulo}");
54                 PaginaActual = paginaAnterior;
55                 Console.WriteLine($"[Cargando] -> {PaginaActual}");
56             }
57             else
58             {
59                 Console.WriteLine($"\\n[Alerta] No hay p ginas en el historial para
60                     retroceder.");
61             }
62         }
63
64         // M todo de Reporter a
```

```

63     public void MostrarHistorial()
64     {
65         Console.WriteLine("\n--- Historial de Navegaci n (Reporte r a) ---");
66         if (historialAtras.Count == 0)
67         {
68             Console.WriteLine("El historial est vac o.");
69             Console.WriteLine("-----\n");
70             return;
71         }
72
73         int posicion = 1;
74         foreach (var pagina in historialAtras)
75         {
76             Console.WriteLine($"{posicion}. {pagina}");
77             posicion++;
78         }
79         Console.WriteLine("-----\n");
80     }
81 }
82
83 class Program
84 {
85     static void Main(string[] args)
86     {
87         Stopwatch cronometro = new Stopwatch();
88         cronometro.Start();
89
90         Navegador miNavegador = new Navegador();
91
92         Console.WriteLine("=== INICIO DE SESI N DEL NAVEGADOR ===");
93
94         miNavegador.VisitarPagina(new PaginaWeb("www.google.com", "Buscador
95             Google"));
96         miNavegador.VisitarPagina(new PaginaWeb("www.github.com", "GitHub Inicio"
97             ));
98         miNavegador.VisitarPagina(new PaginaWeb("www.stackoverflow.com", "
99             StackOverflow"));
100
101         miNavegador.MostrarHistorial();
102
103         miNavegador.Retroceder();
104         miNavegador.MostrarHistorial();
105         miNavegador.Retroceder();
106         miNavegador.Retroceder();
107
108         cronometro.Stop();
109
110         Console.WriteLine("\n=== RESULTADOS DE RENDIMIENTO ===");
111         Console.WriteLine($"Tiempo de ejecuci n total: {cronometro.
112             ElapsedMilliseconds} ms");
113         Console.WriteLine($"Tiempo total en Ticks de CPU: {cronometro.
114             ElapsedTicks}");
115     }
116 }

```